

Python file to be generated; the name of the generated makefile is customisable with the `-m` switch, etc. Finally, you invoke the generated *Makefile* to build the compiled executable from the generated C++ source.

The Shedskin project page provides many test programs in a tar archive. The page mentions a speed-up of 2 to 200 times over CPython, for these test programs. Let's test some of these Python programs ourselves, to check the speed-up. Download the *shedskin-examples-version.tgz* from its page, and run `tar xzvf shedskin-examples-version.tgz && cd shedskin-examples-version`.

First, we test a JPEG-to-BMP conversion program; run `shedskin -m jpg2bmp.mk TonyJpegDecoder.py && make -f jpg2bmp.mk` to build the native executable for this Python program. Now compare the performance of the Python version and the compiled version: run `time python TonyJpegDecoder.py` and `time ./TonyJpegDecoder` and compare the timings. In my test, the compiled version of the *TonyJpegDecoder* sample provided a 1.5x speed-up over the Python version on my Ubuntu machine. I also tried a few other test programs; the compiled version of *chess.py* showed a 12x speed-up, and that of *sudoku1.py* provided a 31x speed-up over their Python versions. Try to measure Shedskin speed-up by experimenting with various kinds of Python programs yourself. You could further explore Shedskin by going through the documentation provided on its project page, and studying the test programs.

Wirbel, a Python-like compiled language

Wirbel is another interesting experimental project for people who are looking to boost Python program performance to near-native speed. It is the fastest of the modern programming languages, as mentioned on its home page. Wirbel is like Shedskin regarding the static data types it uses, and it also uses type inference to deduce information about the data types, to produce the native executable code. Wirbel is a new project with limited library support, so it currently doesn't support the wide functionality that Python does—but it could be useful for small programs in Python-like syntax, to reap the benefits of native-code speed of execution.

To install Wirbel on your machine, download the latest source tarball from its home page, and run `tar xzvf wirbel-version.tar.gz && cd wirbel-version`. Next, run `./configure && make` to build Wirbel. If the compilation completes without errors, then issue `make install` with `sudo` or `su` rights to install it. On my Ubuntu 64-bit machine, I got some compilation failures due to the removal of some headers in the latest versions of g++. So you add the headers `#include <stdint.h>` in both *src/Type.cc* and *src/Location.cc*, and `#include <stdint.h>` in *baustones/http/HTTPRequest.h* respectively. These subdirectories are in the top-level Wirbel source directory. I've mailed this fix to the author of Wirbel, and hope that it'll be included in the next release of the project.

Now you should have two new commands available: the Wirbel compiler, *wic*, and the *wirbel* utility, which compiles

and links the *.w* code and runs the created native executable. The *wic* compiler takes many switches (explore the man page (*man wic*) regarding this). You could add the absolute path to the *wirbel* binary with the *hash bang* (*#!*) magic as the first line of the *.w* files, and make them executable with `chmod +x`, to directly run the scripts.

In addition to the static nature of data types, Wirbel has a few more differences from Python that you should keep in mind while converting existing Python programs:

- The extension of Wirbel source files is *.w* not *.py*; if you try to compile *.py* files with the Wirbel compiler, it throws an error.
- You don't use *import* statements in Wirbel files—it doesn't understand the *import* statement. Wirbel itself finds everything it requires.
- Wirbel supports function overloading, which Python does not.
- The *print* function in Wirbel uses parentheses, like in Python 3.0.

Wirbel comes with many examples and tests that you could go through to see it in action. Measure the performance boost to compare it to similar programs in Python. You can also go through the information on the Wirbel home page to explore it further.

Psyco boosts the performance of Python programs very significantly, and you only need to add a few lines of code. Unladen Swallow is the new turbo-charged Python runtime that optimises CPython without breaking backward compatibility. Shedskin and Wirbel are new experimental projects that target a static subset of Python code to provide native-compiled performance.

In the next part, we will dirty our hands with more options that specifically address the GIL issue in CPython, to extract performance gains for Python programs running on mighty multi-core processors. 

Resources

- Mind-blowing Python GIL: <http://blip.tv/file/2232410>
- Psyco project Page: <http://psyco.sourceforge.net/>
- Unladen Swallow project page: <http://code.google.com/p/unladen-swallow/>
- Boehm garbage collector home page: http://www.hpl.hp.com/personal/Hans_Boehm/gc/
- PCRE library home page: <http://www.pcre.org/>
- Shedskin project page: <http://code.google.com/p/shedskin/>
- Wirbel home page: http://mathias-kettner.de/wirbel_index.html

By: Ankur Kumar Sharma

The author is a software developer and researcher with over 8 years of industry experience in various areas like kernel/hardware programming, system development, automation tools development, R&D, etc. He's the founder of RichNusGeeks Consulting, an open source consulting and training start-up. He enjoys playing classic rock songs on his guitar, likes reading self-help books, writing, and exploring interesting things in his spare time. He blogs at www.richnusgeeks.wordpress.com.